

Lab Sheet: 06026212 DATA WAREHOUSING (การสร้างคลังข้อมูล)

รายวิชา: Data Warehouse

สัปดาห์ที่ 2: การดำเนินการ OLAP ด้วย Metabase และ PostgreSQL

ระยะเวลา: 2 ชั่วโมง

วัตถุประสงค์

- เข้าใจแนวคิด OLAP และ Data Warehouse
- เรียนรู้ SQL Query เพื่อทำ CUBE operations
- ทดลองแสดงผลผ่านแพลตฟอร์ม Metabase ร่วมกับ PostgreSQL
- ปูพื้นฐานการวิเคราะห์แบบ OLAP ก่อนเข้าสู่แนวคิด Star Schema ในสัปดาห์ที่ 4

เครื่องมือที่ใช้

- Metabase (<http://localhost:3000>)
- PostgreSQL
- Database: **dvdrental**
- pgAdmin (สำหรับตรวจสอบฐานข้อมูลด้วย GUI แทนการใช้ CLI ได้)

Part 1: ติดตั้งและนำเข้าฐานข้อมูล dvdrental

1. ดาวน์โหลดไฟล์ dvdrental.tar จาก classroom
(ข้อมูลจาก <https://neon.com/postgresql/getting-started/sample-database>)
2. เปิด Terminal และรันคำสั่งต่อไปนี้:

```
# คัดลอกไฟล์เข้า Container
```

```
docker cp dvdrental.tar dw_postgres:/tmp
```

```
# เข้าไปใน Container
```

```
docker exec -it dw_postgres bash
```

```
# สร้างฐานข้อมูล (ใช้ฐานข้อมูล postgres ในการเชื่อมต่อก่อน)
```

```
psql -U dw_user -d postgres -c "CREATE DATABASE dvdrental;"
```

```
# นำเข้าข้อมูลจาก .tar ไฟล์ (ทางเลือกที่แนะนำ: ใช้ --no-owner)
```

```
pg_restore -U dw_user -d dvdrental \  
--no-owner --role=dw_user /tmp/dvdrental.tar
```

อธิบายคำสั่ง:

- `docker cp` = คัดลอกไฟล์จากเครื่องของเราลง container
- `docker exec -it` = เข้า shell ของ container
- `psql -U dw_user -d postgres -c` = ใช้ผู้ใช้ `dw_user` เชื่อมต่อฐานข้อมูล postgres และรันคำสั่งสร้างฐานข้อมูลใหม่
- `pg_restore` = ใช้คืนค่าฐานข้อมูลจาก backup file
 - `--no-owner` = ไม่พยายามคืนค่า owner เดิมจากระบบอื่น
 - `--role=dw_user` = กำหนด owner ใหม่ให้เป็น `dw_user`

✅ ทางเลือก GUI: หากไม่ถนัด CLI ให้ใช้ **pgAdmin**

- สร้าง database ชื่อ `dvdrental`
 - เลือกเมนู Restore → Format: TAR → เลือกไฟล์ `dvdrental.tar`
 - เลือก Role name เป็น `dw_user`
 - Data Options เลือก Do not save → Owner
 - ตรวจสอบผลลัพธ์ตารางที่เมนู Schemas → Tables
4. ตรวจสอบตารางในฐานข้อมูล (ผ่าน CLI):

```
psql -U dw_user -d dvdrental -c "\dt"
```

อธิบายคำสั่ง:

- `\dt` = แสดงรายการตารางทั้งหมดในฐานข้อมูล

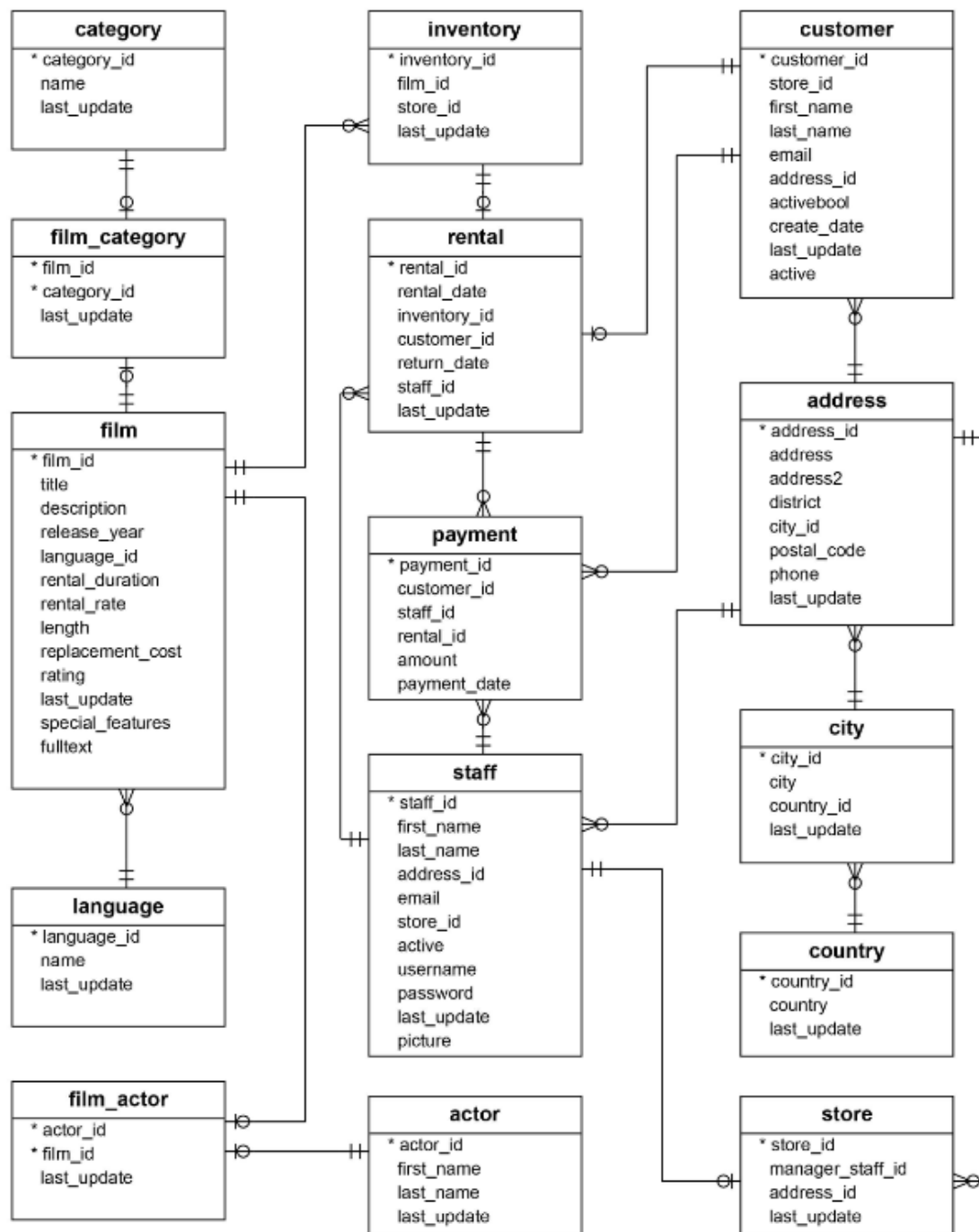
Part 2: เชื่อมต่อ Metabase กับ PostgreSQL

- เปิด Metabase ที่ <http://localhost:23000>
- เลือก "Add your data" → PostgreSQL
- กรอกค่าดังนี้:
 - Display name: `dvdrental`
 - Hostname: `dw_postgres`
 - Database name: `dvdrental`
 - Username: `dw_user`
 - Password: `dw_pass`

Part 3: สืบหา Table

- ดู Table เช่น: `film`, `category`, `customer`, `rental`, `payment`
- สังเกตความสัมพันธ์ของข้อมูลแบบ Dim-Fact เบื้องต้น

คำอธิบายโครงสร้าง `dvdrental`:



ตาราง	ประเภท	เหตุผล
payment	Fact	เก็บข้อมูลการจ่ายเงินที่มี timestamp และจำนวน
customer	Dimension	ใช้ระบุว่าลูกค้าคนใดเป็นผู้จ่าย
staff	Dimension	ใช้ดูว่าใครเป็นคนรับชำระเงิน
rental	Fact	เก็บข้อมูลการเช่าหนัง

ตาราง	ประเภท	เหตุผล
film	Dimension	ข้อมูลรายละเอียดภาพยนตร์
category	Dimension	หมวดหมู่ของหนัง
inventory	Bridge	เชื่อมระหว่าง film และ rental
store	Dimension	ร้านค้าที่ลูกค้าใช้บริการ
city / address	Dimension	บริบทสถานที่ของลูกค้า

💡 ใน OLAP การจัดระเบียบข้อมูลเป็น Fact และ Dimension จะช่วยให้การวิเคราะห์แบบ Roll-up/Drill-down มีประสิทธิภาพมากขึ้น

Part 4: ทดลองเขียน SQL Query (ผ่าน SQL Editor)

เปิด SQL Editor ใน Metabase → เลือก dvdrental แล้วทดลองรัน SQL ต่อไปนี้:

◆ Roll-up: เมือง (City) → ประเทศ (Country)

```
SELECT co.country, ci.city, SUM(p.amount) AS total_revenue
FROM payment p
JOIN customer cu ON p.customer_id = cu.customer_id
JOIN address a ON cu.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
GROUP BY ROLLUP (co.country, ci.city)
ORDER BY co.country, ci.city;
```

- ◆ Drill-down: ลงไปดูตามเขต (district) และชื่อพนักงาน (staff)

```
SELECT co.country, a.district, s.first_name AS staff_name,  
       SUM(p.amount) AS total_revenue  
FROM payment p  
JOIN staff s ON p.staff_id = s.staff_id  
JOIN customer cu ON p.customer_id = cu.customer_id  
JOIN address a ON cu.address_id = a.address_id  
JOIN city ci ON a.city_id = ci.city_id  
JOIN country co ON ci.country_id = co.country_id  
GROUP BY co.country, a.district, s.first_name;
```

- ◆ Slice: เอาเฉพาะประเทศ 'Thailand'

```
SELECT ci.city, SUM(p.amount) AS total_revenue  
FROM payment p  
JOIN customer cu ON p.customer_id = cu.customer_id  
JOIN address a ON cu.address_id = a.address_id  
JOIN city ci ON a.city_id = ci.city_id  
JOIN country co ON ci.country_id = co.country_id  
WHERE co.country = 'Thailand'  
GROUP BY ci.city;
```

- ◆ Dice: ระบุเงื่อนไขพร้อมกันทั้งมิติ ประเทศ และ พนักงานผู้รับเงิน

```
SELECT co.country, ci.city, s.first_name AS staff_name,
       SUM(p.amount) AS total_revenue
FROM payment p
JOIN staff s ON p.staff_id = s.staff_id
JOIN customer cu ON p.customer_id = cu.customer_id
JOIN address a ON cu.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
WHERE co.country IN ('Thailand', 'Japan')
      AND s.staff_id = 1
GROUP BY co.country, ci.city, s.first_name;
```

- ◆ CUBE Operation: หาเส้นทางความสัมพันธ์และยอดรวมสลับกันระหว่าง ประเทศที่ลูกค้าอาศัยอยู่ กับ พนักงานที่ให้บริการ ในทุกกรณีที่เป็นไปได้

```
SELECT co.country, s.first_name AS staff_name,
       SUM(p.amount) AS total_revenue
FROM payment p
JOIN staff s ON p.staff_id = s.staff_id
JOIN customer cu ON p.customer_id = cu.customer_id
JOIN address a ON cu.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
GROUP BY CUBE (co.country, s.first_name)
ORDER BY co.country, s.first_name;
```



PostgreSQL รองรับ GROUP BY CUBE มาตั้งแต่เวอร์ชัน 9.5 โดยจะสร้าง aggregation combinations เช่น (country, first_name), (country), (first_name), และ ()



Part 5: สร้าง Dashboard ด้วย Metabase (สำหรับผู้เริ่มต้น)

- ◆ ขั้นตอนที่ 1: เปิด SQL Editor

- เข้าสู่หน้าแรก Metabase
- คลิกเมนูด้านซ้าย → Databases → dvdrental → +New → SQL query

- วาง SQL Query แล้วกด Run
- ◆ ขั้นตอนที่ 2: เปลี่ยนผลลัพธ์เป็นกราฟ
 - คลิก Visualization → เลือก Chart Type
 - ปรับแต่ง SQL query และ chart options ตามต้องการ
- ◆ ขั้นตอนที่ 3: บันทึกกราฟ
 - คลิก Save → ตั้งชื่อกราฟ เช่น drill-down
 - เลือก Save to new dashboard → ตั้งชื่อ Dashboard เช่น:

OLAP Analysis - [ชื่อของคุณ]

- ◆ ขั้นตอนที่ 4: ปรับ Layout
 - เข้าเมนู Dashboards → เปิด Dashboard
 - จัดวางกราฟให้เหมาะสมและอ่านง่าย



หมายเหตุ:

- สามารถสร้าง Dashboard แบบ Drag-and-drop ได้จาก Interface โดยไม่ต้องเขียน SQL ก็ได้ในภายหลัง
- นักศึกษาสามารถใช้ Metabase GUI เพื่อสร้าง Query ใหม่โดยเลือกตารางและเขตข้อมูลได้โดยตรง



สิ่งที่ต้องส่งใน Google Form

ตอบคำถามใน Google Classroom หัวข้อ Lab 2: การดำเนินการ OLAP ด้วย Metabase และ PostgreSQL

หมายเหตุ: หากมีปัญหาในการนำเข้า dvdrental ให้แจ้งอาจารย์ หรือ TA แทนที่